



第4章 常微分方程数值解法

计算机学院 孙伟平



第4章

4.0 引言（常微分方程初值问题及其解法概述）

4.1 欧拉方法

- 显式欧拉法、隐式欧拉法、二步欧拉法、梯形公式欧拉法
- 局部截断误差

4.2 改进的欧拉方法

4.3 龙格-库塔方法

4.4 单步法的收敛性与稳定性

4.0 引言

- 在自然科学与工程技术的许多领域中,经常会遇到常微分方程定解问题。
- 本章主要以一阶常微分方程为主,介绍常微分方程初值问题的差分方法和相关理论。

一阶常微分方程初值问题的一般形式:

$$\begin{cases} y'(x) = f(x, y) & \text{微分方程} \\ y(x_0) = y_0 & \text{初始条件} \end{cases}$$

其中 $f(x, y)$ 是已知函数, y_0 为给定的值。

1. 何时存在唯一解?
2. 如何计算 $y(x)$?


$$\begin{cases} y' = f(x, y) \iff y'(x) = f(x, y(x)) \\ y(x_0) = y_0 \end{cases}$$

定理： 若 $f(x, y)$ 在某闭区域 R :

$$|x - x_0| \leq a, \quad |y - y_0| \leq b \quad (a > 0, b > 0)$$

上连续, 且在 R 域内关于 y 满足李普希兹 (Lipschitz) 条件, 即存在正数 L , 使得对于 R 域内的任意两值 y_1, y_2 , 下列不等式成立:

$$|f(x, y_1) - f(x, y_2)| \leq L |y_2 - y_1|$$

则上述初值问题的连续可微的解 $y(x)$ 存在并且唯一。



- ◆ 实际生产与科研中，除少数简单情况能获得初值问题的初等解（用初等函数表示的解）外，绝大多数情况下是求不出初等解的。
- ◆ 有些初值问题即便有初等解，也往往由于形式过于复杂而不便处理。
- ◆ 实用的方法是在计算机上进行数值求解：即不直接求 $y(x)$ 的显式解，而是在解所存在的区间上，求得 $y(x)$ 在一系列点 x_n ($n = 1, 2, \dots$) 上的近似值。

◆ 约定:

- $y(x_n)$: 待求函数 $y(x)$ 在 x_n 处的精确函数值
- y_n : 待求函数 $y(x)$ 在 x_n 处的近似函数值

$$\begin{cases} y' = f(x, y) & y'(x_n) = f(x_n, y(x_n)) \quad n = 1, 2, \dots \\ y(x_0) = y_0 \end{cases}$$

- ◆ 本章的任务: 计算出解函数 $y(x)$ 在一系列节点 $a = x_0 < x_1 < \dots < x_N = b$ 处的近似值 $y_n \approx y(x_n)$

建立常微分方程数值方法的基本思想

微分方程数值解法，其实是求出方程的解 $y(x)$ 在一系列离散点上的近似值。微分方程数值解的基本思想是：求解区间和方程离散化。

➤ 求解区间离散化

将求解区间 $[a, b]$ 离散化，是在 $[a, b]$ 上插入一系列的分点 $\{x_k\}$ ，使 $a = x_0 < x_1 < \dots < x_n < x_{n+1} < \dots < x_N = b$

记 $h_n = x_{n+1} - x_n$ ($n = 0, 1, \dots, N-1$) 称为步长。

一般取 $h_n = h$ ，其中 $h = \frac{b-a}{N}$ ，称节点

$$x_n = x_0 + nh \quad (n = 0, 1, 2, \dots, N)$$

为等步长节点。



➤ 将微分方程离散化

(1) 差商逼近法

即用适当的差商近似导数值。

(2) 数值积分法

基本思想是先将问题转化为积分方程

$$y(x_m) - y(x_n) = \int_{x_n}^{x_m} f(x, y(x)) dx \quad (y(x_0) = y_0)$$

然后将上式右端采用上一章介绍的数值积分离散化，从而获得原初值问题的一个离散差分格式。

(3) Taylor展开法

见后面的叙述。

补充

一元Taylor公式

$$y(x_{n+1}) = y(x_n + h) = y(x_n) + y'(x_n)h + \frac{y''(x_n)}{2!}h^2 + \frac{y'''(x_n)}{3!}h^3 + O(h^4)$$

二元Taylor公式

$$\begin{aligned} f(x_n + h, y_n + k) &= f(x_n, y_n) + \frac{\partial f(x_n, y_n)}{\partial x} h + \frac{\partial f(x_n, y_n)}{\partial y} k \\ &+ \frac{1}{2!} \left[\frac{\partial^2 f(x_n, y_n)}{\partial x^2} h^2 + 2 \frac{\partial^2 f(x_n, y_n)}{\partial x \partial y} hk + \frac{\partial^2 f(x_n, y_n)}{\partial y^2} k^2 \right] + \square \\ &+ \frac{1}{k!} \left[h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right]^k f(x_n, y_n) + \square \quad \square \end{aligned}$$

4.1 欧拉（Euler）方法

莱昂哈德·欧拉（Leonhard Euler，1707年～1783年），瑞士数学家、自然科学家。

- 18世纪数学界最杰出的人物之一，他不但为数学界作出贡献，更把整个数学推至物理的领域。
- 力学、分析学、几何学、变分法等 的课本，《无穷小分析引论》、《微分学原理》、《积分学原理》等都成为数学界中的经典著作。此外欧拉还涉及建筑学、弹道学、航海学等领域。
- 欧拉是科学史上最多产的一位杰出的数学家，据统计他那不倦的一生，共写下了886本书籍和论文。



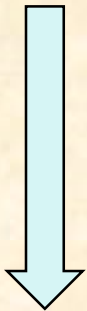
瑞士法郎正面的Euler肖像

——摘自百度百科

4.1.1 欧拉法

$$\begin{cases} y' = f(x, y) & a \leq x \leq b \\ y(x_0) = y_0 \end{cases} \quad (1)$$

改写成
递推式



$$\begin{cases} y_{n+1} = \varphi(x_n, y_n, h), & n = 0, 1, 2, \dots \\ y(x_0) = y_0 \end{cases}$$

假设初值问题(1)的解 $y=y(x)$ 唯一存在且足够光滑. 对求解区域 $[a, b]$ 做等距剖分

$$a=x_0 < x_1 < x_2 < \dots < x_n < \dots < x_N=b$$

$h=(b-a)/N$ 称为剖分步长; 剖分节点

$$x_n = a + nh, \quad n=0, 1, \dots, N.$$

数值解法的目标就是求精确解 $y(x)$ 在剖分节点 x_n 上的值 $y(x_n)$ 的近似值 $y_n, n=1, 2, \dots, N$.

1. 四种欧拉方法
2. 局部截断误差


$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

◆ 方法一 化导数为差商的方法

1. 向前差商数值微分公式

$$y'(x_n) \approx \frac{y(x_n + h) - y(x_n)}{h}$$

$$y'(x_n) \approx \frac{y(x_{n+1}) - y(x_n)}{h}$$

2. 向后差商数值微分公式

$$y'(x_n) \approx \frac{y(x_n) - y(x_n - h)}{h}$$

$$y'(x_n) \approx \frac{y(x_n) - y(x_{n-1})}{h}$$

3. 中心差商数值微分公式

$$y'(x_n) \approx \frac{y(x_n + h) - y(x_n - h)}{2h}$$

$$y'(x_n) \approx \frac{y(x_{n+1}) - y(x_{n-1}))}{2h}$$


$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases} \quad y'(x_n) = f(x_n, y(x_n))$$

$$y'(x_n) \approx \frac{y(x_{n+1}) - y(x_n)}{h}$$

$$\frac{y(x_{n+1}) - y(x_n)}{h} = f(x_n, y(x_n))$$

$$y(x_{n+1}) = y(x_n) + hf(x_n, y(x_n))$$

◆ 由于在逐步求解的过程中， $y(x_n)$ 的准确值无法求解出来，因此用其近似值 y_n 代替。

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) \\ y_0 = y(x_0) \end{cases} \quad n = 0, 1, 2, \dots$$

Euler方法
显式
单步


$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases} \quad y'(x_n) = f(x_n, y(x_n))$$

$$y'(x_n) \approx \frac{y(x_n) - y(x_{n-1}))}{h}$$

$$\frac{y(x_n) - y(x_{n-1}))}{h} = f(x_n, y(x_n))$$

$$y(x_n) = y(x_{n-1}) + hf(x_n, y(x_n))$$

$$\begin{cases} y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) \\ y_0 = y(x_0) \end{cases} \quad n = 0, 1, 2, \dots$$

Euler方法
隐式
单步


$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases} \quad y'(x_n) = f(x_n, y(x_n))$$

$$y'(x_n) \approx \frac{y(x_{n+1}) - y(x_{n-1}))}{2h}$$

$$\frac{y(x_{n+1}) - y(x_{n-1}))}{2h} = f(x_n, y(x_n))$$

$$y(x_{n+1}) = y(x_{n-1}) + 2hf(x_n, y(x_n))$$

$$\begin{cases} y_{n+1} = y_{n-1} + 2hf(x_n, y_n) \\ y_0 = y(x_0) \end{cases} \quad n = 1, 2, \dots$$

Euler方法
显式
二步

$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

◆ 方法二 数值积分法

将微分方程 $y' = f(x, y)$ 在区间 $[x_n, x_{n+1}]$ 上积分：

$$y(x_{n+1}) - y(x_n) = \int_{x_n}^{x_{n+1}} y'(x) dx = \int_{x_n}^{x_{n+1}} f(x, y(x)) dx$$

左矩形公式

$$\int_a^b f(x) dx \approx (b - a) f(a)$$

右矩形公式

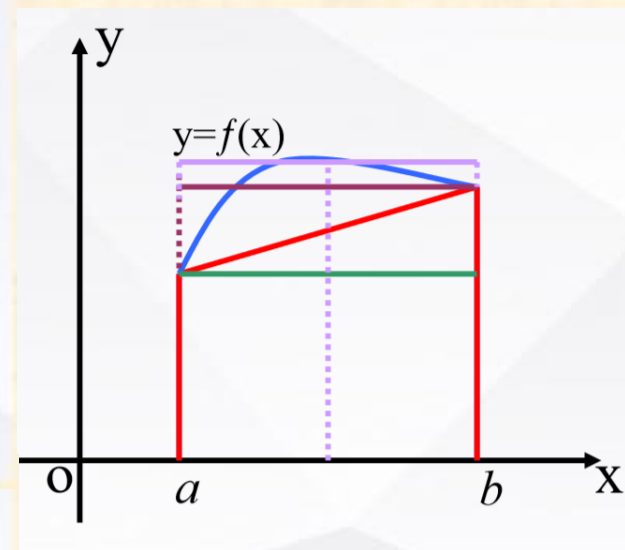
$$\int_a^b f(x) dx \approx (b - a) f(b)$$

中矩形公式

$$\int_a^b f(x) dx \approx (b - a) f\left(\frac{a+b}{2}\right)$$

梯形公式

$$\int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)]$$



显式欧拉法

将微分方程 $y' = f(x, y)$ 在区间 $[x_n, x_{n+1}]$ 上积分:

$$\begin{aligned} y(x_{n+1}) - y(x_n) &= \int_{x_n}^{x_{n+1}} y' \, dx = \int_{x_n}^{x_{n+1}} f(x, y(x)) \, dx \\ &\approx (x_{n+1} - x_n) f(x_n, y(x_n)) \quad \text{左矩形公式} \\ &= hf(x_n, y(x_n)) \end{aligned}$$

同样以近似值 y_n 代替精确值 $y(x_n)$ 可得:

Euler法
显式
单步

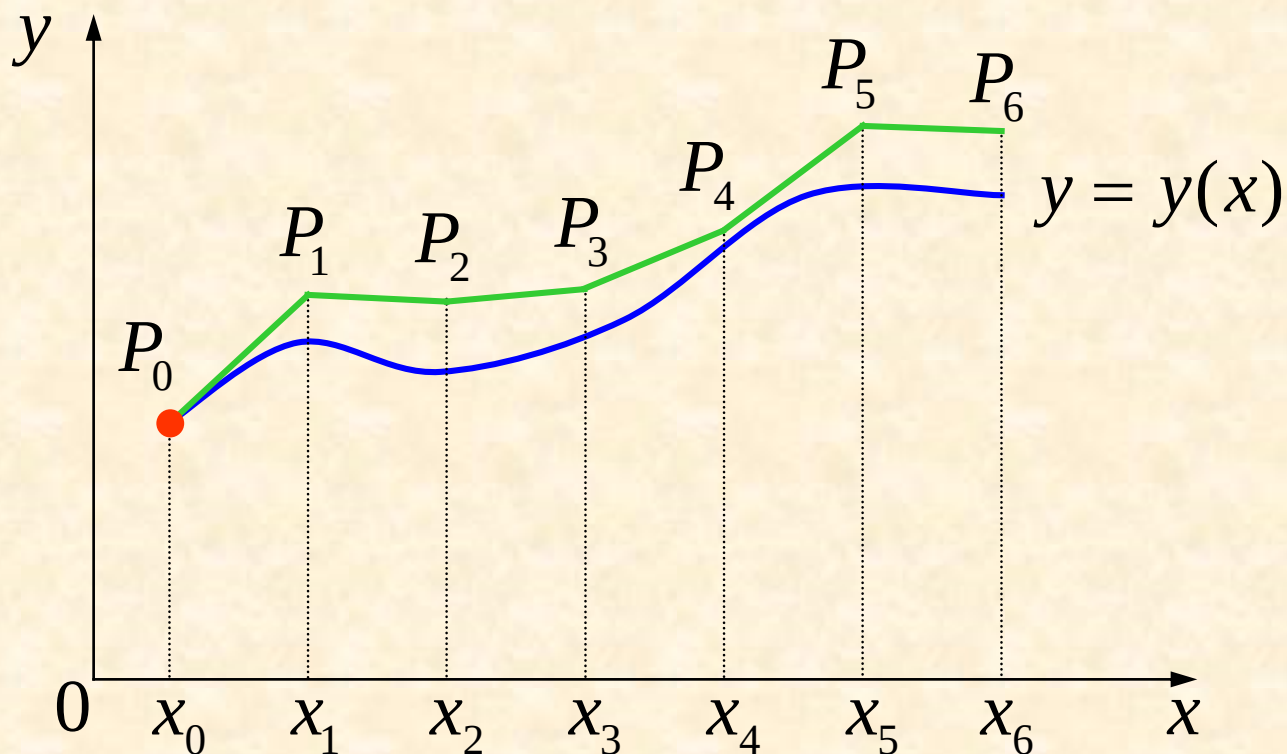
$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) \\ y_0 = y(x_0) \end{cases}$$


$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) & n = 0, 1, 2, \dots \\ y_0 = y(x_0) \end{cases}$$

- ◆ 根据 y_0 可以一步步计算出函数 $y = y(x)$ 在 $x_1, x_2, x_3, x_4, \dots$ 上的近似值 $y_1, y_2, y_3, y_4, \dots$
- ◆ 常微分方程数值解是一组离散的函数值数据，它的精确表达式很难求解得到，但可以进行插值计算后用插值函数逼近 $y(x)$

显式欧拉法的几何意义

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) & n = 0, 1, 2, \dots \\ y_0 = y(x_0) \end{cases}$$



隐式欧拉法

$$\begin{aligned} y(x_{n+1}) - y(x_n) &= \int_{x_n}^{x_{n+1}} y' \, dx = \int_{x_n}^{x_{n+1}} f(x, y(x)) \, dx \\ &\approx (x_{n+1} - x_n) f(x_{n+1}, y(x_{n+1})) \quad \text{右矩形公式} \\ &= hf(x_{n+1}, y(x_{n+1})) \end{aligned}$$

◆ 用近似值代替精确值可得：

Euler法
隐式
单步

$$\begin{cases} y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) \\ y_0 = y(x_0) \end{cases}$$

含有未知
的函数值

◆ 隐式欧拉法没有显式欧拉法方便。

二步欧拉法

- 在数值积分法推导中，积分区间宽度选为两步步长，即积分区间为： $[x_{n-1}, x_{n+1}]$ ，则：

$$\begin{aligned} y(x_{n+1}) - y(x_{n-1}) &= \int_{x_{n-1}}^{x_{n+1}} y' dx = \int_{x_{n-1}}^{x_{n+1}} f(x, y(x)) dx \\ &\approx (x_{n+1} - x_{n-1}) f(x_n, y(x_n)) \quad \text{中矩形公式} \\ &= 2h f(x_n, y(x_n)) \end{aligned}$$

- 以 $y(x)$ 在 x_{n-1}, x_n 上的近似值代替精确值可得：

Euler方法
显式
二步

$$\begin{cases} y_{n+1} = y_{n-1} + 2h f(x_n, y_n) \\ y_0 = y(x_0) \end{cases}$$

需要前两步
的计算结果

梯形公式欧拉法

- 在数值积分法中，如果用梯形公式近似计算 $f(x, y)$ 在区间 $[x_n, x_{n+1}]$ 上的积分，即：

$$\int_{x_n}^{x_{n+1}} f(x, y(x)) dx \approx (x_{n+1} - x_n) \cdot \frac{f(x_n, y(x_n)) + f(x_{n+1}, y(x_{n+1}))}{2}$$
$$= \frac{h}{2} [f(x_n, y(x_n)) + f(x_{n+1}, y(x_{n+1}))]$$

- 用近似值代替精确值可得梯形公式欧拉法：

隐式
单步

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})] \\ y_0 = y(x_0) \end{cases}$$

- 上式右端出现了未知项，可见梯形法是隐式欧拉法的一种；实际上，梯形公式欧拉法是显式欧拉法与隐式欧拉法的算术平均。

例 题

用显式欧拉法、隐式欧拉法、梯形法求解初值问题：

$$\begin{cases} y' = -y + x + 1 \\ y(0) = 1 \end{cases}$$

取 $h = 0.1$ ，计算到 $x = 0.5$ ，并与精确解进行比较。

解：由已知条件可得： $h = 0.1$ ， $x_0 = 0$ ， $y_0 = 1$ ，

$$f(x, y) = -y + x + 1$$

◆ 显式欧拉法：

$$\begin{aligned} y_{n+1} &= y_n + h(-y_n + x_n + 1) \\ &= (1 - h)y_n + hx_n + h \\ &= 0.9y_n + 0.1x_n + 0.1 \end{aligned}$$



◆ 显式欧拉法: $y_{n+1} = 0.9y_n + 0.1x_n + 0.1$

◆ 隐式欧拉法: $y_{n+1} = y_n + h(-y_{n+1} + x_{n+1} + 1)$

化简得: $(1+h)y_{n+1} = y_n + h(x_{n+1} + 1) = y_n + h(x_n + h + 1)$

$$y_{n+1} = \frac{1}{(1+h)}[y_n + h(x_n + h + 1)] = (y_n + 0.1x_n + 0.11)/1.1$$
$$(\approx 0.909y_n + 0.091x_n + 0.1)$$

◆ 梯形公式欧拉法:

$$y_{n+1} = y_n + \frac{h}{2}[(-y_n + x_n + 1) + (-y_{n+1} + x_{n+1} + 1)]$$

$$[1 + (h/2)]y_{n+1} = [1 - (h/2)]y_n + (h/2)(x_n + x_{n+1} + 2)$$

$$y_{n+1} = [(2-h)y_n + h(x_n + x_{n+1} + 2)]/(2+h)$$

$$= (1.9y_n + 0.2x_n + 0.21)/2.1$$

$$(\approx 0.904y_n + 0.095x_n + 0.1)$$



显式法	$y_{n+1} = 0.9y_n + 0.1x_n + 0.1$
隐式法	$y_{n+1} = (y_n + 0.1x_n + 0.11)/1.1$
梯形法	$y_{n+1} = (1.9y_n + 0.2x_n + 0.21)/2.1$

◆ 计算结果:

x_n	显式法 y_n	隐式法 y_n	梯形法 y_n	精确解 $y(x_n)$
0.0	1	1	1	1
0.1	1.000000	1.009091	1.004762	1.004837
0.2	1.010000	1.026446	1.018594	1.019731
0.3	1.029000	1.051315	1.040633	1.040818
0.4	1.056100	1.083014	1.070097	1.070320
0.5	1.090490	1.120922	1.106278	1.106531

本题的精确解为: $y(x) = x + e^{-x}$

4.1.2 局部截断误差与精度

- 在一阶常微分方程初值问题的数值求解中，当利用某一递推公式计算出 $y_1, y_2, y_3, \dots$ 时，每一步都会产生误差。若计算到 x_{n+1} ，则误差为

$$\varepsilon_{n+1} = y(x_{n+1}) - y_{n+1}$$

称为方法在 x_{n+1} 点的整体截断误差。

- 现假设 $y_n = y(x_n)$ ，即在前一步 y_n 准确的前提下，估计：

$$T_{n+1} = y(x_{n+1}) - y_{n+1}$$

称其为该数值计算方法在 x_{n+1} 点的局部截断误差。

- 
- ◆ 如果某个常微分方程数值算法的局部截断误差可表示为 $O(h^{p+1})$ ，则称该数值算法的精度是 p 阶。精度阶数越高，方法的精度越好。
 - ◆ 可证明Euler法、隐式Euler法、二步Euler法、梯形公式Euler法的精度分别是一阶、一阶、二阶和二阶，即它们的局部截断误差分别是 $O(h^2)$ 、 $O(h^2)$ 、 $O(h^3)$ 和 $O(h^3)$ 。
 - ◆ 分析局部截断误差/精度，用到泰勒展开式。

局部截断误差的分析

- ◆ 如果初值问题中的 $f(x, y)$ 充分可微，则可将 $y(x_{n+1})$ 在点 x_n 处展开：

$$\begin{aligned} y(x_{n+1}) &= y(x_n + h) \\ &= \underbrace{y(x_n)}_{y_n} + h \underbrace{y'(x_n)}_{f(x_n, y_n)} + \frac{h^2}{2!} \underbrace{y''(x_n)}_{f_x + f_y y'} + \cdots + \underbrace{O(h^{p+1})}_{\boxed{\frac{h^{p+1}}{(p+1)!} y^{(p+1)}(\xi)}} \end{aligned}$$

- ◆ 利用泰勒公式展开，比较各算法与展开式的前几项

$$y(x_{n+1}) - y_{n+1}$$

由算法得到

将 $y(x_{n+1})$ 在 x_n 点处用泰勒公式展开:

$$y(x_{n+1}) = \underbrace{y(x_n)}_{y_n} + h \underbrace{y'(x_n)}_{f(x_n, y_n)} + \frac{h^2}{2!} \underbrace{y''(x_n)}_{f_x + f_y y'} + \cdots + \frac{h^{p+1}}{(p+1)!} y^{(p+1)}(\xi)$$

$$y_{n+1} = y_n + hf(x_n, y_n)$$

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$$

$$y_{n+1} = y_{n-1} + 2hf(x_n, y_n)$$

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$$

$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{y^{(p+1)}(\xi)}{(p+1)!} h^{p+1} = O(h^{p+1}) \quad \text{p 阶精度}$$



$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{y^{(p+1)}(\xi)}{(p+1)!} h^{p+1} = O(h^{p+1})$$

$$y(x_{n+1}) = \underbrace{y(x_n)}_{y_n} + h \underbrace{y'(x_n)}_{f(x_n, y_n)} + \frac{h^2}{2!} \underbrace{y''(x_n)}_{f_x + f_y y'} + \dots + \frac{h^{p+1}}{(p+1)!} y^{(p+1)}(\xi)$$

$$y_{n+1} = y_n + hf(x_n, y_n) \quad \checkmark$$

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$$

$$y_{n+1} = y_{n-1} + 2hf(x_n, y_n)$$

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$$



$$y(x_{n+1}) = \underbrace{y(x_n)}_{y_n} + h \underbrace{y'(x_n)}_{f(x_n, y_n)} + \frac{h^2}{2!} y''(x_n) + \cdots \frac{h^{p+1}}{(p+1)!} y^{(p+1)}(\xi)$$

对隐式公式的处理:

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$$

$$\underline{f(x_{n+1}, y_{n+1})} \approx \underline{f(x_{n+1}, y(x_{n+1}))} = y'(x_{n+1})$$

$$\text{对 } y' \text{ 做一次泰勒展开} \quad = y'(x_n) + O(h)$$

➤ 显式欧拉法的局部截断误差:

欧拉法 $y_{n+1} = y_n + hf(x_n, y_n)$

令: $y_n = y(x_n)$

$$y_{n+1} = y(x_n) + hf(x_n, y(x_n)) = y(x_n) + hy'(x_n)$$

$$y(x_{n+1}) = y(x_n + h) = y(x_n) + hy'(x_n) + \frac{y''(\xi)}{2!} h^2$$
$$\xi \in (x_n, x_{n+1})$$

$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{y''(\xi)}{2!} h^2 = O(h^2) \quad \text{1 阶精度}$$


$$y(x_n)$$

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$$

➤ 隐式欧拉法的局部截断误差:

由泰勒公式 $y(x_{n+1}) = y(x_n + h) = \underline{y(x_n)} + hy'(x_n) + O(h^2)$ (1)

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) \quad (2)$$

令隐式欧拉法右边的 $y_{n+1} = y(x_{n+1})$

$$\begin{aligned} y_{n+1} &= y_n + hf(x_{n+1}, y(x_{n+1})) \\ &= y_n + hy'(x_{n+1}) \\ &= y_n + h(y'(x_n) + O(h)) \\ &= y_n + hy'(x_n) + O(h^2) \quad (3) \end{aligned}$$

(1)-(3)得 $T_{n+1} = y(x_{n+1}) - y_{n+1} = O(h^2)$

1 阶精度

$$y_{n+1} = y_{n-1} + 2hf(x_n, y_n)$$

➤ 二步欧拉法的局部截断误差:

$$y(x_{n-1})$$

$$y(x_n)$$

$$y_{n+1} = y(x_{n-1}) + 2hf(x_n, y(x_n)) = y(x_{n-1}) + 2hy'(x_n)$$

分别将 $y(x_{n+1})$, $y(x_{n-1})$ 在 x_n 点处用泰勒公式展开:

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{y''(x_n)}{2!}h^2 + O(h^3)$$

$$y(x_{n-1}) = y(x_n) - hy'(x_n) + \frac{y''(x_n)}{2!}(-h)^2 + O(h^3)$$

$$T_{n+1} = y(x_{n+1}) - y_{n+1}$$

$$= y(x_{n+1}) - y(x_{n-1}) - 2hy'(x_n)$$

$$= O(h^3)$$

2 阶精度

梯形公式欧拉法:

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$$

求证梯形公式欧拉法的精度为2阶。

(1) 泰勒公式 $y(x_{n+1}) = y(x_n + h) = y(x_n) + hy'(x_n) + \frac{h^2}{2!} y''(x_n) + O(h^3)$

(2) $y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$

(3) 令 $y_n = y(x_n)$, 令隐式欧拉法右边的 $y_{n+1} = y(x_{n+1})$

(4)
$$\begin{aligned} y_{n+1} &= y(x_n) + (h/2)(y'(x_n) + y'(x_{n+1})) \\ &= y(x_n) + (h/2)[y'(x_n) + (y'(x_n) + hy''(x_n) + O(h^2))] \\ &= y(x_n) + hy'(x_n) + (h^2/2)y''(x_n) + O(h^3) \end{aligned}$$

(5) $T_{n+1} = y(x_{n+1}) - y_{n+1} = O(h^3)$

2 阶精度

四种欧拉法的比较

方法	显式/隐式	单步/多步	精度	评述
欧拉法	显式	单步	1	最简单，精度低
隐式欧拉法	隐式	单步	1	不便计算，稳定性好 (*)
二步欧拉法	显式	多步	2	需要两步初值，且第 2 个初值只能由其它方法给出，可能对后面的递推精度有影响
梯形公式欧拉法	隐式	单步	2	精度有所提高，但为隐式，需要迭代求解，计算量大

鱼和熊掌可兼得乎？

4.2 改进的欧拉法

- ◆ 从上述例子可以看到，梯形法由于具有二阶精度，其局部截断误差比显式欧拉法和隐式欧拉法小，但梯形法是一种隐式算法，需要迭代求解，计算量大。
- ◆ 显式欧拉法是一个显式算法，虽然计算量较小，但是精度不高。
- ◆ 综合两种方法的长处，可以先用显式欧拉法求出 $y(x_{n+1})$ 的一个粗略近似值，然后用它代入梯形法公式的右端，用梯形法计算 $y(x_{n+1})$ 的较为精确的近似值。

预报值 $\bar{y}_{n+1}$

校正值 y_{n+1}

改进的欧拉法（续）

显式欧拉法 $y_{n+1} = y_n + hf(x_n, y_n)$

梯形法 $y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$

◆ 按照上述思想，可以建立如下预报-校正系统：

预报： $\bar{y}_{n+1} = y_n + hf(x_n, y_n)$

校正： $y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})]$

◆ 按以上两式求解常微方程的算法称为改进的欧拉法。

◆ 改进的欧拉法的精度是二阶。（证明？）

改进的欧拉法（续）

预报： $\bar{y}_{n+1} = y_n + h f(x_n, y_n)$

校正： $y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})]$

◆ 改进的欧拉法还可以表示为：

嵌套形式

$$y_{n+1} = y_n + \frac{h}{2} \left\{ f(x_n, y_n) + f(x_{n+1}, y_n + h f(x_n, y_n)) \right\}$$

平均化形式

$$\begin{cases} y_{n+1} = (y_p + y_c) / 2 \\ y_p = y_n + h f(x_n, y_n) \\ y_c = y_n + h f(x_{n+1}, y_p) \end{cases}$$

◆ 改进的欧拉法的平均化形式:

$$\begin{cases} y_{n+1} = (y_p + y_c)/2 \\ y_p = y_n + h f(x_n, y_n) \\ y_c = y_n + h f(x_{n+1}, y_p) \end{cases}$$

$$\begin{cases} y_{n+1} = y_n + h(\frac{1}{2}k_1 + \frac{1}{2}k_2) \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + h, y_n + hk_1) \end{cases}$$

— 用于分析几何意义

$$\begin{cases} y_{n+1} = y_n + (\frac{1}{2}k_1 + \frac{1}{2}k_2) \\ k_1 = hf(x_n, y_n) \\ k_2 = hf(x_n + h, y_n + k_1) \end{cases}$$

— 用于扩展


$$\begin{cases} y' = -y + x + 1 \\ y(0) = 1 \end{cases} \quad h = 0.1, \text{ 计算到 } x = 0.5$$

例 用改进欧拉法求上例所述的初值问题，并与欧拉法和梯形法比较误差的大小。

解：采用改进欧拉法的嵌套形式：

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{2} \{ \underline{f}(x_n, y_n) + \underline{f}(x_{n+1}, y_n + h f(x_n, y_n)) \} \\ &= y_n + \frac{h}{2} \{ (-y_n + x_n + 1) + \{-[y_n + h \underline{f}(x_n, y_n)] + x_{n+1} + 1\} \} \\ &= y_n + \frac{h}{2} \{ (-y_n + x_n + 1) - [y_n + h(-y_n + x_n + 1)] + (x_n + h) + 1 \} \\ &= \left[1 + \frac{h(-2 + h)}{2} \right] y_n + \frac{h(2 - h)}{2} x_n + h = 0.905 y_n + 0.095 x_n + 0.1 \end{aligned}$$

$$\begin{cases} y' = -y + x + 1 \\ y(0) = 1 \end{cases} \quad h = 0.1, \quad \text{计算到 } x = 0.5$$

◆ 计算结果（误差-绝对误差）

x_n	改进欧拉法 y_n	精确解 $y(x_n)$	误 差		
			改进欧拉法	欧拉法	梯形法
0.1	1.005000	1.004837	1.6×10^{-4}	4.8×10^{-3}	7.5×10^{-5}
0.2	1.019025	1.019731	7.1×10^{-4}	8.7×10^{-3}	1.4×10^{-4}
0.3	1.041218	1.040818	4.0×10^{-4}	1.2×10^{-2}	1.9×10^{-4}
0.4	1.070802	1.070320	4.8×10^{-4}	1.4×10^{-2}	2.2×10^{-4}
0.5	1.107076	1.106531	5.5×10^{-4}	1.6×10^{-2}	2.5×10^{-4}

◆ 可见，改进欧拉法的误差数量级与梯形法大致相同，而比欧拉法小得多。

深入理解改进的欧拉法

改进的欧拉法的平均化形式

$$\begin{cases} y_p = y_n + h f(x_n, y_n) \\ y_c = y_n + h f(x_{n+1}, y_p) \\ y_{n+1} = (y_p + y_c)/2 \end{cases}$$

k_1 points to $f(x_n, y_n)$
 $y_n + hk_1$ points to y_p
 k_2 points to $f(x_{n+1}, y_p)$


$$y_{n+1} = y_n + \frac{h}{2}k_1 + \frac{h}{2}k_2 = y_n + h \frac{k_1 + k_2}{2}$$

◆ $y(x_{n+1})$ 在点 x_n 处的一阶展开式为:

$$y(x_{n+1}) = y(x_n + h) = y(x_n) + h \underline{y'(\xi)} \quad \xi \in (x_n, x_{n+1})$$

A red arrow points from the boxed term $\frac{k_1 + k_2}{2}$ in the previous equation to the underlined term $y'(\xi)$ in this equation.

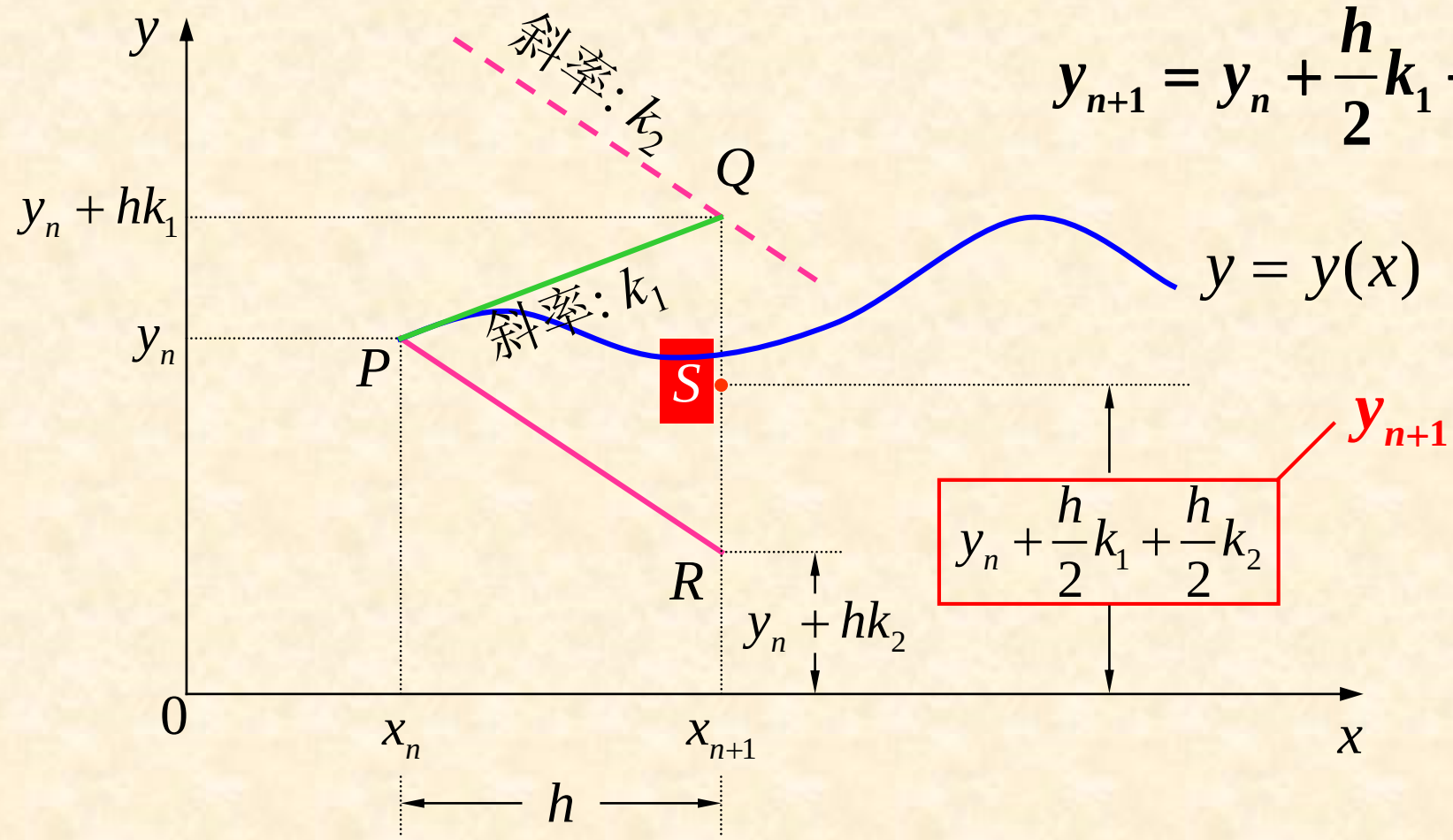


$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

$$k_1 = f(x_n, y_n)$$

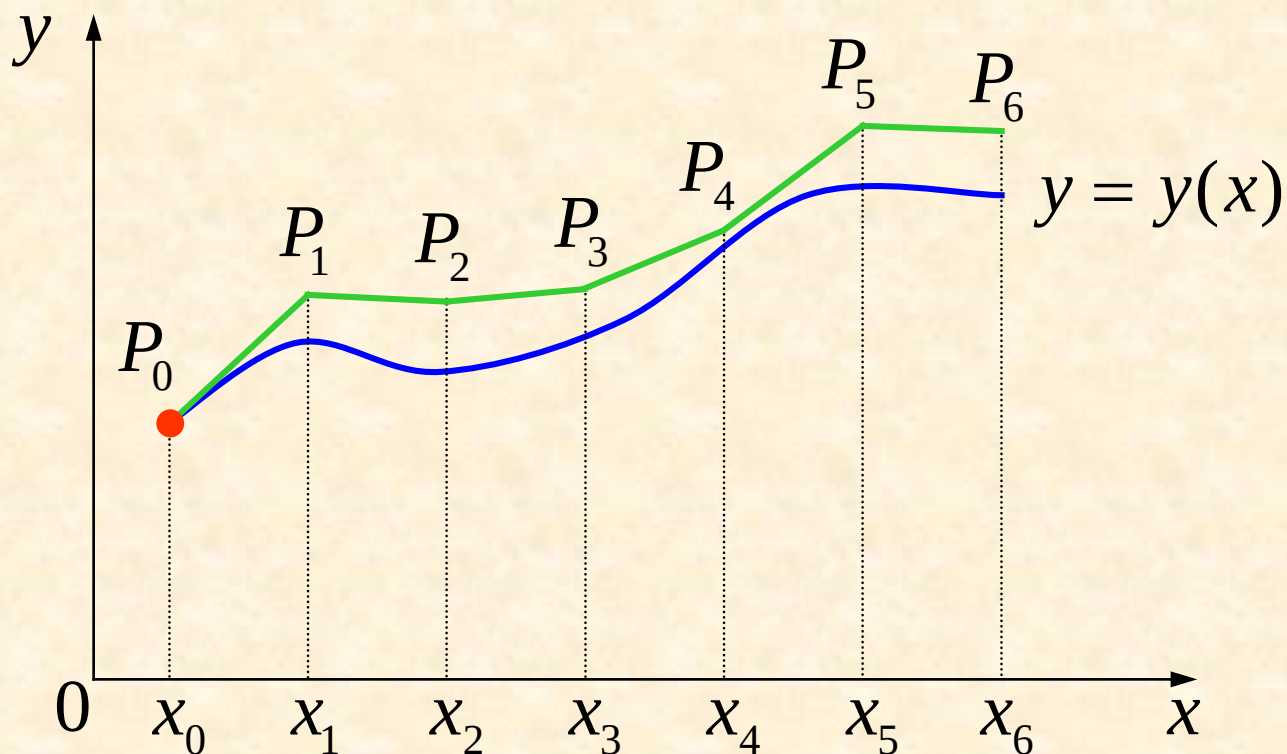
$$k_2 = f(x_{n+1}, y_n + hk_1)$$

$$y_{n+1} = y_n + \frac{h}{2}k_1 + \frac{h}{2}k_2$$



显式欧拉法的几何意义

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) & n = 0, 1, 2, \dots \\ y_0 = y(x_0) \end{cases}$$



4.3 龙格-库塔(Runge-Kutta)方法

◆ 显式欧拉法 (1 阶精度)

$$y_{n+1} = y_n + hf(x_n, y_n)$$



$$\begin{cases} y_{n+1} = y_n + k_1 \\ k_1 = hf(x_n, y_n) \end{cases}$$

◆ 改进的欧拉法 (2 阶精度)

$$\begin{cases} y_p = y_n + hf(x_n, y_n) \\ y_c = y_n + hf(x_{n+1}, y_p) \\ y_{n+1} = (y_p + y_c)/2 \end{cases}$$



$$\begin{cases} y_{n+1} = y_n + \frac{1}{2}(k_1 + k_2) \\ k_1 = hf(x_n, y_n) \\ k_2 = hf(x_{n+1}, y_n + k_1) \end{cases}$$

◆ $y(x_{n+1})$ 在点 x_n 处的一阶泰勒展开式为:

$$y(x_{n+1}) = y(x_n + h) = y(x_n) + hy'(\xi)$$

$$= y(x_n) + hf(\xi, y(\xi)) \quad \xi \in (x_n, x_{n+1})$$

k^*



- ◆ 显式欧拉法用一个点的值 k_1 作为 k^* 的近似值
- ◆ 改进的欧拉公式用两个点的值 k_1 和 k_2 的平均值作为 k^* 近似值；
- ◆ 改进的欧拉法比显式欧拉法精度高；
- ◆ 在 $[x_n, x_{n+1}]$ 内多预报几个点的 k_i 值，并用其加权平均值作为 k^* 的近似值，从而构造出具有**更高精度**的计算公式，这就是**龙格-库塔方法的基本思想**。

p阶R-K公式

$$\left\{ \begin{array}{l} y_{n+1} = y_n + \underline{(c_1 k_1 + c_2 k_2 + \cdots + c_p k_p)} \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + a_2 h, y_n + b_{21} k_1) \\ k_3 = h f(x_n + a_3 h, y_n + b_{31} k_1 + b_{32} k_2) \\ \dots \\ k_p = h f(x_n + a_p h, y_n + \sum_{i=1}^{p-1} b_{pi} k_i) \end{array} \right. \quad \begin{array}{l} \{c_i, a_i, b_{ij}\} \\ \text{为待定系数} \end{array}$$

如果此公式的局部截断误差为 $O(h^{p+1})$ ，则称其为（p级）p阶 Runge-Kutta公式，简称为（p级）p阶R-K公式。

二阶龙格-库塔方法

$$\begin{cases} y_{n+1} = y_n + c_1 k_1 + c_2 k_2 \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + a_2 h, y_n + b_{21} k_1) \end{cases}$$

c_1, c_2, a_2, b_{21} 为待定系数

求待定系数，以使得该公式的精度尽可能高。

◆ 为分析局部截断误差，令 $y_n = y(x_n)$ ，由泰勒公式得：

$$y(x_{n+1}) = y(x_n + h) = y(x_n) + h y'(x_n) + \frac{h^2}{2!} y''(x_n) + O(h^3)$$

$$y'(x_n) = f(x_n, y(x_n)) = f(x_n, y_n)$$

$$y''(x_n) = f'(x_n, y_n) = f_x(x_n, y_n) + f_y(x_n, y_n) f(x_n, y_n)$$

$$y(x_{n+1}) = y(x_n) + h f(x_n, y_n)$$

$$+ \frac{h^2}{2} [f_x(x_n, y_n) + f_y(x_n, y_n) f(x_n, y_n)] + O(h^3)$$

二阶龙格-库塔方法

$$\begin{cases} y_{n+1} = y_n + c_1 k_1 + c_2 k_2 \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + a_2 h, y_n + b_{21} k_1) \end{cases}$$

c_1, c_2, a_2, b_{21}
为待定系数

$$y(x_{n+1}) = y(x_n) + hf(x_n, y_n) + \frac{h^2}{2} [f_x(x_n, y_n) + f_y(x_n, y_n) f(x_n, y_n)] + O(h^3)$$

$$y_{n+1} = y_n + c_1 k_1 + c_2 k_2$$

$$= y_n + c_1 h f(x_n, y_n) + c_2 \underline{h f(x_n + a_2 h, y_n + b_{21} k_1)}$$

补充：二元泰勒展开式

$$f(x_0 + h, y_0 + k) = f(x_0, y_0)$$

$$+ \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right) f(x_0, y_0)$$

$$+ \frac{1}{2!} \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right)^2 f(x_0, y_0)$$

$\vdots$

$$+ \frac{1}{n!} \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right)^n f(x_0, y_0)$$

$+ \dots$

$$h f_x(x_0, y_0) + k f_y(x_0, y_0)$$

$$h^2 f_{xx}(x_0, y_0) + 2hk f_{xy}(x_0, y_0) + k^2 f_{yy}(x_0, y_0)$$

$$\sum_{i=0}^n C_n^i h^i k^{n-i} \frac{\partial^n f}{\partial x^i \partial y^{n-i}} \bigg|_{\substack{x=x_0 \\ y=y_0}}$$

$$y(x_{n+1}) = y(x_n) + hf(x_n, y_n)$$

$$k_1 = hf(x_n, y_n)$$

$$+ \frac{h^2}{2} [f_x(x_n, y_n) + f_y(x_n, y_n)f(x_n, y_n)] + O(h^3)$$

◆ 用二元泰勒公式展开 $k_2 = hf(x_n + a_2h, y_n + b_{21}k_1)$

$$k_2 = h[f(x_n, y_n) + a_2hf_x(x_n, y_n) + b_{21}k_1f_y(x_n, y_n) + O(h^2)]$$

◆ 将 k_1, k_2 代入 $y_{n+1} = y_n + c_1k_1 + c_2k_2$ 中可得:

$$y_{n+1} = y_n + c_1hf(x_n, y_n) + c_2h[f(x_n, y_n) + a_2hf_x(x_n, y_n) + b_{21}k_1f_y(x_n, y_n)] + O(h^3)$$

$$= y_n + (c_1 + c_2)hf(x_n, y_n) + c_2a_2h^2f_x(x_n, y_n) + c_2b_{21}h^2f_y(x_n, y_n)f(x_n, y_n) + O(h^3)$$

$$= y_n + (c_1 + c_2)hf(x_n, y_n) + \frac{h^2}{2} [2c_2a_2f_x(x_n, y_n) + 2c_2b_{21}f_y(x_n, y_n)f(x_n, y_n)] + O(h^3)$$

二阶龙格-库塔方法（续）

$$y_{n+1} = y_n + (c_1 + c_2)h f(x_n, y_n) + \frac{h^2}{2} [2c_2 a_2 f_x(x_n, y_n) + 2c_2 b_{21} f_y(x_n, y_n) f(x_n, y_n)] + O(h^3)$$

$$y(x_{n+1}) = y(x_n) + h f(x_n, y_n) + \frac{h^2}{2} [f_x(x_n, y_n) + f_y(x_n, y_n) f(x_n, y_n)] + O(h^3)$$

$$\begin{cases} c_1 + c_2 = 1 \\ 2c_2 a_2 = 1 \\ 2c_2 b_{21} = 1 \end{cases} \longrightarrow y(x_{n+1}) - y_{n+1} = O(h^3) \quad \text{2 阶精度}$$

$$c_1 + c_2 = 1$$

$$2c_2a_2 = 1$$

$$2c_2b_{21} = 1$$

$$\begin{cases} y_{n+1} = y_n + c_1k_1 + c_2k_2 \\ k_1 = hf(x_n, y_n) \\ k_2 = hf(x_n + a_2h, y_n + b_{21}k_1) \end{cases}$$

◆ 四个未知变量，只有三个方程，有无穷多组解

◆ 每组解的构成的龙格-库塔方法均为二阶

$$\text{取} \begin{cases} c_1 = c_2 = 1/2 \\ a_2 = b_{21} = 1 \end{cases} \begin{cases} y_{n+1} = y_n + (\frac{1}{2}k_1 + \frac{1}{2}k_2) \\ k_1 = hf(x_n, y_n) \\ k_2 = hf(x_n + h, y_n + k_1) \end{cases}$$

改进的欧拉法

$$\text{取} \begin{cases} c_1 = 0 \\ c_2 = 1 \\ a_2 = b_{21} = 1/2 \end{cases} \begin{cases} y_{n+1} = y_n + k_2 \\ k_1 = hf(x_n, y_n) \\ k_2 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right) \end{cases}$$

变形的欧拉法
中点方法

三阶龙格-库塔方法

◆ 三阶龙格-库塔方法是用三个值 k_1, k_2, k_3 的加权平均来近似取代 k^*

$$\begin{cases} y_{n+1} = y_n + c_1 k_1 + c_2 k_2 + c_3 k_3 \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + a_2 h, y_n + b_{21} k_1) \\ k_3 = h f(x_n + a_3 h, y_n + b_{31} k_1 + b_{32} k_2) \end{cases}$$

◆ 要使三阶龙格-库塔方法具有三阶精度，必须使其局部截断误差为 $O(h^4)$

◆ 将 k_1, k_2, k_3 代入 y_{n+1} 的表达式中，在 (x_n, y_n) 处用二元泰勒公式展开，与 $y(x_{n+1})$ 在 x_n 处的泰勒展开式比较⁵⁵

三阶龙格-库塔方法（续）

- ◆ 类似二阶龙格-库塔方法的推导过程，8 个待定系数 $c_1, c_2, c_3, a_2, a_3, b_{21}, b_{31}, b_{32}$ 应满足：

$$\left\{ \begin{array}{l} c_1 + c_2 + c_3 = 1 \\ a_2 = b_{21} \\ a_3 = b_{31} + b_{32} \\ c_2 a_2 + c_3 a_3 = 1/2 \\ c_2 a_2^2 + c_3 a_3^2 = 1/3 \\ c_2 b_{32} a_2 = 1/6 \end{array} \right.$$

8 个未知参数，6 个方程，
有无穷多组解

库塔公式

$$\left\{ \begin{array}{l} y_{n+1} = y_n + \frac{1}{6}k_1 + \frac{4}{6}k_2 + \frac{1}{6}k_3 \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1) \\ k_3 = h f(x_n + h, y_n - k_1 + 2k_2) \end{array} \right.$$

四阶龙格-库塔方法

- ◆ 类似可以推出四阶龙格-库塔公式，常用的有：

经典四阶龙格-库塔公式

$$\left\{ \begin{array}{l} y_{n+1} = y_n + \frac{1}{6}k_1 + \frac{2}{6}k_2 + \frac{2}{6}k_3 + \frac{1}{6}k_4 \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1) \\ k_3 = h f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_2) \\ k_4 = h f(x_n + h, y_n + k_3) \end{array} \right.$$

四阶龙格-库塔方法（续）

吉尔（Gill）公式

$$\left\{ \begin{array}{l} y_{n+1} = y_n + \frac{1}{6}k_1 + \frac{2-\sqrt{2}}{6}k_2 + \frac{2+\sqrt{2}}{6}k_3 + \frac{1}{6}k_4 \\ k_1 = h f(x_n, y_n) \\ k_2 = h f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1) \\ k_3 = h f(x_n + \frac{1}{2}h, y_n + \frac{\sqrt{2}-1}{2}k_1 + \frac{2-\sqrt{2}}{2}k_2) \\ k_4 = h f(x_n + h, y_n - \frac{\sqrt{2}}{2}k_2 + \frac{2+\sqrt{2}}{2}k_3) \end{array} \right.$$

- ◆ 四阶以上龙格-库塔公式的计算量太大，并且精度不一定提高，有时反而会降低，因此实际应用中一般选用四阶龙格-库塔已足可满足精度要求。


$$\begin{cases} y' = -y + x + 1 \\ y(0) = 1 \end{cases} \quad h = 0.1, \text{ 计算到 } x = 0.5$$

用经典四阶龙格-库塔方法求解前例的初值问题，并与改进的欧拉法、梯形法在 $x_5 = 0.5$ 处比较其误差大小。

解: $y_{n+1} = y_n + \frac{1}{6}k_1 + \frac{2}{6}k_2 + \frac{2}{6}k_3 + \frac{1}{6}k_4$

$$y_0 \Rightarrow y_1$$

$$k_1 = hf(x_0, y_0) = 0.1 \times (-y_0 + x_0 + 1) = 0$$

$$\begin{aligned} k_2 &= hf\left(x_0 + \frac{1}{2}h, y_0 + \frac{1}{2}k_1\right) \\ &= 0.1 \times [-(y_0 + \frac{1}{2}k_1) + (x_0 + \frac{1}{2}h) + 1] = 0.005 \end{aligned}$$

$$\begin{aligned} k_3 &= hf\left(x_0 + \frac{1}{2}h, y_0 + \frac{1}{2}k_2\right) \\ &= 0.1 \times [-(y_0 + \frac{1}{2}k_2) + (x_0 + \frac{1}{2}h) + 1] = 0.00475 \end{aligned}$$

$$\begin{aligned} k_4 &= hf(x_0 + h, y_0 + k_3) \\ &= 0.1 \times [-(y_0 + k_3) + (x_0 + h) + 1] = 0.009525 \end{aligned}$$


$$\begin{cases} y' = -y + x + 1 \\ y(0) = 1 \end{cases} \quad h = 0.1, \text{ 计算到 } x = 0.5$$

于是: $y_1 = y_0 + \frac{1}{6}k_1 + \frac{2}{6}k_2 + \frac{2}{6}k_3 + \frac{1}{6}k_4$

$$= 1 + \frac{1}{6} \times 0 + \frac{2}{6} \times 0.005 + \frac{2}{6} \times 0.00475 + \frac{1}{6} \times 0.009525$$
$$= 1.0048375$$

同理 $y_2 = 1.0187309$

可计 $y_3 = 1.04081842$

算: $y_4 = 1.07032029$

$$y_5 = 1.1065309$$

精确解为:

$$y(x) = x + e^{-x}$$

$$y(x_5) = 0.5 + e^{-0.5} \approx 1.106530660$$

四阶R-K方法的
精度比二阶方法
高得多

R-K方法的误差: $|y(x_5) - y_5| = 2.4 \times 10^{-7}$

改进欧拉法的误差: 5.5×10^{-4}

梯形法的误差: 2.5×10^{-4}

变步长的龙格-库塔方法

- ◆ **R-K方法求解初值问题**，其精度与步长 h 有关。那么步长 h 该如何选取呢？
- ◆ 如果在整个区间上统一用大步长可能达不到精度要求，而使用小步长会增加计算量，还会导致舍入误差累积的增加。
- ◆ 希望能根据解的性态来调整步长的大小：在变化平缓的部分，数值求解使用较大步长；而在变化剧烈的部分，则使用较小步长。这样就能在保证精度的前提下尽可能减少计算量。
- ◆ 因此有必要讨论变步长的**R-K方法**。下面以变步长的四阶**R-K方法**为例。

变步长的龙格-库塔方法

- ◆ 设 $y(x_n)$ 在 x_n 处的值 $y_n = y(x_n)$ ，当 $x_{n+1} = x_n + h$ 时 $y(x_{n+1})$ 的近似值为 $y_{n+1}^{(h)}$ ，由于四阶 R-K 方法的精度为 4 阶，故局部截断误差为：

$$T_{n+1} = y(x_{n+1}) - y_{n+1}^{(h)} \approx c_n h^5$$

- ◆ 若以 $h/2$ 为步长，从 x_n 出发，经过两步计算，得到 $y(x_{n+1})$ 的近似值 $y_{n+1}^{(h/2)}$
- ◆ 以上每步的截断误差约为 $c_n(h/2)^5$ ，于是两步的局部截断误差为：

$$T_{n+1} = y(x_{n+1}) - y_{n+1}^{(h/2)} \approx 2c_n (h/2)^5$$

变步长的龙格-库塔方法（续）

于是：

$$\frac{y(x_{n+1}) - y_{n+1}^{(h/2)}}{y(x_{n+1}) - y_{n+1}^{(h)}} \approx \frac{2c_n(h/2)^5}{c_n h^5} = \frac{1}{16}$$

整理得：

$$y(x_{n+1}) - y_{n+1}^{(h/2)} \approx \frac{1}{15} [y_{n+1}^{(h/2)} - y_{n+1}^{(h)}]$$

记 $\Delta = \left| \frac{1}{15} [y_{n+1}^{(h/2)} - y_{n+1}^{(h)}] \right|$ ，设给定的精度要求为 ε

- ◆ 若 $\Delta > \varepsilon$ ，则将步长反复折半，直至 $\Delta < \varepsilon$ ；
- ◆ 若 $\Delta < \varepsilon$ ，则停止。（或将步长反复加倍，直至 $\Delta > \varepsilon$ 这时再将步长折半一次；）
- ◆ 取最终得到的 $y_{n+1}^{(h/2)}$ 作为 $y(x_{n+1})$ 的近似值。

变步长的龙格-库塔方法（续）

- 从节点 x_n 出发，求在节点 x_{n+1} 处的近似值。
- 给定精度 ε 。

1. 选取一步长 h ，用龙格-库塔方法计算 $y_{n+1}^{(h)}$ ；
2. 将步长折半，以步长 $h/2$ 从 x_n 用龙格-库塔方法计算两次，跨两步到节点 x_{n+1} ，得到 $y_{n+1}^{(h/2)}$ ；
3. 计算 $\Delta = \left| \frac{1}{15} [y_{n+1}^{(h/2)} - y_{n+1}^{(h)}] \right|$
 - ◆ 若 $\Delta < \varepsilon$ ，则停止
 - ◆ 若 $\Delta > \varepsilon$ ，重复步骤2、3
4. 取最终得到的 $y_{n+1}^{(h/2)}$ 作为 $y(x_{n+1})$ 的近似值。

4.4 单步法的收敛性与稳定性

- ◆ 常微分方程初值问题的数值方法是经过某种离散化过程导出的，差分解能否作为精确解的近似？需要对其进行定性分析。
- ◆ 收敛性和稳定性是从不同的角度来考量算法的可靠性。
- ◆ 本节只考虑单步显式方法的收敛性和单步法的稳定性。而且稳定性的分析只针对特定模型的单步法进行。
- ◆ 重点
 - 定义
 - 判定定理及应用

单步法的收敛性

◆ 求解初值问题

$$\begin{cases} y'(x) = f(x, y), a \leq x \leq b \\ y(x_0) = y_0 \end{cases}$$

的单步显式法可统一写成如下形式：

$$\varphi(x, y, h)$$

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h)$$

称为增量函数，依赖于函数 f ，是 x, y, h 的函数

例如对于欧拉方法，有 $\varphi(x, y, h) = f(x, y)$

对于改进的欧拉方法，有

$$\varphi(x, y, h) = \frac{1}{2} [f(x, y) + f(x + h, y + hf(x, y))]$$



◆ 显式单步法可统一写成:

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h)$$

$\varphi(x, y, h)$

求 $y = y(x)$

离散化

求 $y(x_n)$, $x_n = x_0 + nh$

某种数值方法

求 $y_n \approx y(x_n)$

对任意固定的 $x_n = x_0 + nh$,
当 $h \rightarrow 0$ 时, 近似解 y_n
是否收敛到精确解 $y(x_n)$?

$x_n = x_0 + nh$ 是一个固定节点, 因此当 $h \rightarrow 0$ 时应同时附带 $n \rightarrow \infty$

单步法的收敛性（续）

- ◆ 对于 p 阶精度的常微分方程初值问题数值算法，当 $h \rightarrow 0$ ， $n \rightarrow \infty$ 时，是否 $y_n \rightarrow y(x_n)$ ？

- ◆ p 阶精度算法的局部截断误差为：

$$y(x_n) - y_n = O(h^{p+1})$$

显然：

$$\lim_{h \rightarrow 0, n \rightarrow \infty} [y(x_n) - y_n] = 0$$

- ◆ 局部截断误差的前提假设是： $y_{n-1} = y(x_{n-1})$
- ◆ 局部截断误差 $\rightarrow 0$ 并不能保证算法收敛。

单步法的收敛性（续）

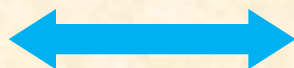
定义： 求解某初值问题的单步数值法，对任意固定的 $x_n = x_0 + nh$ ，当 $h \rightarrow 0$ 且 $n \rightarrow \infty$ 时，它的近似解趋近于精确解 $y(x_n)$ ，即：

$$\lim_{h \rightarrow 0, n \rightarrow \infty} y_n = y(x_n)$$

则称该单步法是收敛的。

定义： 称 $|y(x_n) - y_n|$ 为单步法的近似解 y_n 的**整体截断误差**。

单步法收敛



$h \rightarrow 0$ 且 $n \rightarrow \infty$ 时， y_n 的整体截断误差 $\rightarrow 0$

收敛性定理

(定理4.2)

若单步法 $y_{n+1} = y_n + h\varphi(x_n, y_n, h)$ 满足以下条件:

- ◆ 具有 p 阶精度 (即局部截断误差为 $O(h^{p+1})$) ;
- ◆ 增量函数 $\varphi(x, y, h)$ 关于 y 满足 Lipschitz 条件:

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L_\varphi |y - \bar{y}|$$

- ◆ 初值 y_0 是准确的。

则该单步法的整体截断误差为: $|y(x_n) - y_n| = O(h^p)$

若某单步法满足以上条件, 则该方法是收敛的。

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h)$$

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L_\varphi |y - \bar{y}|$$

证明：假设在 **前一步** y_n **准确** 的前提下求得的近似值为：

$$\bar{y}_{n+1} = y(x_n) + h\varphi(x_n, y(x_n), h)$$

ε_{n+1}

算法精度为 p 阶，局部截断误差：
$$|\bar{y}_{n+1} - y(x_{n+1})| \leq ch^{p+1}$$

$$|y(x_{n+1}) - y_{n+1}| = |y(x_{n+1}) - \bar{y}_{n+1}| + |\bar{y}_{n+1} - y_{n+1}|$$

$$\leq |y(x_{n+1}) - \bar{y}_{n+1}| + |\bar{y}_{n+1} - y_{n+1}|$$

$$\leq ch^{p+1} + |\{y(x_n) - y_n\} + \{h\varphi(x_n, y(x_n), h) - h\varphi(x_n, y_n, h)\}|$$

$$\leq ch^{p+1} + |y(x_n) - y_n| + h|\varphi(x_n, y(x_n), h) - \varphi(x_n, y_n, h)|$$

$$\leq ch^{p+1} + |y(x_n) - y_n| + hL_\varphi |y(x_n) - y_n|$$

$$= ch^{p+1} + (1 + hL_\varphi) |y(x_n) - y_n| \quad \varepsilon_n$$



$$\varepsilon_{n+1} = (1 + hL_\varphi)\varepsilon_n + ch^{p+1}$$

$$\begin{aligned}
 \varepsilon_n &\leq (1 + hL_\varphi)\varepsilon_{n-1} + ch^{p+1} \\
 &\leq (1 + hL_\varphi)\left[(1 + hL_\varphi)\varepsilon_{n-2} + ch^{p+1}\right] + ch^{p+1} \\
 &= (1 + hL_\varphi)^2\varepsilon_{n-2} + [(1 + hL_\varphi) + 1] \cdot ch^{p+1} \\
 &\leq (1 + hL_\varphi)^2\left[(1 + hL_\varphi)\varepsilon_{n-3} + ch^{p+1}\right] + [(1 + hL_\varphi) + 1] \cdot ch^{p+1} \\
 &= (1 + hL_\varphi)^3\varepsilon_{n-3} + [(1 + hL_\varphi)^2 + (1 + hL_\varphi) + 1] \cdot ch^{p+1} \\
 &\vdots \\
 &= (1 + hL_\varphi)^n\varepsilon_0 + [(1 + hL_\varphi)^{n-1} + \dots + (1 + hL_\varphi) + 1] \cdot ch^{p+1} \\
 &= (1 + hL_\varphi)^n\varepsilon_0 + \frac{1 \cdot [(1 + hL_\varphi)^n - 1]}{(1 + hL_\varphi) - 1} ch^{p+1} \\
 &= (1 + hL_\varphi)^n\varepsilon_0 + \left[(1 + hL_\varphi)^n - 1\right] \frac{ch^p}{L_\varphi}
 \end{aligned}$$


$$\varepsilon_n \leq (1 + hL_\varphi)^n \varepsilon_0 + \left[(1 + hL_\varphi)^n - 1 \right] \frac{ch^p}{L_\varphi}$$

$$e^x = 1 + x + \frac{e^\xi}{2} x^2 \geq 1 + x$$

即: $(1 + x)^n \leq (e^x)^n$

◆ $y = e^x$ 为单调增函数, 当 $x_n - x_0 = nh \leq T$ 时

$$(1 + hL_\varphi)^n \leq \left(e^{hL_\varphi} \right)^n = e^{nhL_\varphi} \leq e^{TL_\varphi}$$

$$\varepsilon_n \leq (1 + hL_\varphi)^n \varepsilon_0 + \frac{ch^p}{L_\varphi} (e^{TL_\varphi} - 1)$$

◆ 若初值是准确的, 则 $\varepsilon_0 = 0$, 从而整体截断误差为:

$$\varepsilon_n \leq \frac{ch^p}{L_\varphi} (e^{TL_\varphi} - 1) \quad O(h^p)$$

◆ 当 $h \rightarrow 0$ 且 $n \rightarrow \infty$ 时, 则 $\varepsilon_n \rightarrow 0$

例 题

如果 f 关于 y 满足 Lipschitz 条件,考察改进的Euler法的收敛性。

参见教材

验证其它单步法（含经典四阶R-K方法）的收敛性。

自行验证

注意：验证单步法的收敛性，首先要将公式转化成如下形式：

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h)$$

按照**收敛性定理**，逐条验证。

单步法的稳定性

- ◆ 在讨论单步法收敛性时一般认为数值方法本身的计算过程是准确的，实际上并非如此：
 - 初始值 y_0 有误差 $\delta = y_0 - y(x_0)$
 - 后续的每一步计算均有可能有舍入误差
- ◆ 这些初始和舍入误差在计算过程的传播中是逐步衰减的还是恶性增长就是数值方法的稳定性问题。
- ◆ 第3章Newton-Cotes公式的稳定性分析的启发：舍入误差的分析复杂，需对问题进行简化。

单步法的稳定性（续）

定义：设在节点 x_n 处用数值算法得到的理想数值解为 y_n ，而实际计算得到的近似解为 $\bar{y}_n$ ，称差值：

$$\delta_n = \bar{y}_n - y_n$$

为第 n 步的数值解的**扰动**。

定义：若一种数值方法在节点 x_n 处的数值解 y_n 的扰动 $\delta_n \neq 0$ ，而在以后各节点 y_m ($m > n$) 上产生的扰动为 δ_m ，如果：

$$|\delta_m| \leq |\delta_n| \quad (m = n+1, n+2, \dots)$$

则称该数值方法是**稳定**的。

单步法的稳定性（续）

数值方法的稳定性一般与步长 h 的取值有关。

有的数值方法是恒稳定的，有的数值方法在一定条件下是稳定的。

- ◆ 由于函数 $f(x, y)$ 的多样性，数值稳定性的分析相当复杂，通常只研究模型方程

$$y' = \lambda y \quad (\lambda < 0)$$

也即只考虑如下的特殊情况：

$$f(x, y) = \lambda y \quad (\lambda < 0)$$

欧拉法的稳定性

➤ 欧拉法: $y_{n+1} = y_n + hf(x_n, y_n)$

考察模型方程:

$$y' = \lambda y \quad (\lambda < 0)$$

即:

$$y_{n+1} = y_n + h(\lambda y_n) = (1 + h\lambda)y_n$$

假设在节点值 y_n 上有扰动 δ_n , 在节点值 y_{n+1} 上有扰动 δ_{n+1} , 且 δ_{n+1} 仅由 δ_n 引起 (即: 计算过程中不再引起新的误差)

针对模型方程: $y' = \lambda y \quad (\lambda < 0)$

显式欧拉法: $y_{n+1} = (1 + h\lambda)y_n$

扰动: $\delta_{n+1} = (1 + h\lambda)\delta_n \quad \delta_{n+m} = (1 + h\lambda)^m \delta_n$

欧拉法稳定 $\longleftrightarrow |\delta_{n+m}| \leq |\delta_n| \longleftrightarrow |1 + h\lambda| \leq 1$

即: $-1 \leq 1 + h\lambda \leq 1$

化简得: $-2 \leq h\lambda \leq 0 \quad \lambda < 0$

欧拉法稳定的条件:

$$0 < h \leq -\frac{2}{\lambda}$$

隐式欧拉法的稳定性

考察模型方程: $y' = \lambda y \quad (\lambda < 0)$

隐式欧拉法 $y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) = y_n + h\lambda y_{n+1}$

化简为: $y_{n+1} = \frac{1}{1-h\lambda} y_n$

假设 y_n 上有扰动 δ_n , 则 y_{n+1} 的扰动为:

$$\delta_{n+1} = \frac{1}{1-h\lambda} \delta_n \quad \delta_{n+m} = \frac{1}{(1-h\lambda)^m} \delta_n$$

隐式欧拉法稳定 $\longleftrightarrow |\delta_{n+m}| \leq |\delta_n| \longleftrightarrow \left| \frac{1}{1-h\lambda} \right| \leq 1 \quad \lambda < 0$

$\forall h > 0$, 上式均成立, 所以

隐式欧拉法是恒稳定的。

其他方法的稳定性

- ◆ 类似地，可分析其他方法对于模型方程的稳定性

$$y' = \lambda y \quad (\lambda < 0)$$

梯形公式欧拉法

恒稳定

经典四阶R-K方法

一定条件下稳定

改进的欧拉法

一定条件下稳定

变形的欧拉法

习题

参见教材



例 考虑初值问题

$$\begin{cases} y' = -30y, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

试取步长 $h=0.1$, 利用欧拉法计算 $y_{10} \approx y(1)$

这个步长取值是否合适?

分析: 欧拉法的稳定条件是 $-2 \leq h\lambda \leq 0$,

此题中 $h\lambda = 0.1 \times (-30) = -3$, 不满足稳定条件。

实际上, 欧拉法经10步计算得到 $y_{10} = 1024$, 而精确值 $y(1) = 9.357623 \times 10^{-14}$

收敛性与稳定性 小结

- ◆ 收敛性 —— 整体截断误差；只考虑单步显式方法的收敛性。

收敛性： $y_{n+1} = y_n + h\varphi(x_n, y_n, h)$

收敛性定理：若满足以下条件：

- ◆ 具有 p 阶精度（局部截断误差 $O(h^{p+1})$ ）；
- ◆ 函数 $\varphi(x, y, h)$ 关于 y 满足 Lipschitz 条件：

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L_\varphi |y - \bar{y}|$$

- ◆ 初值是准确的。

则该方法是收敛的。

- 
- ◆ 稳定性 —— 数值解的扰动，稳定性分析只考虑单步方法，并针对特定模型进行，即

$$y' = \lambda y \quad (\lambda < 0)$$

确定方法中 h 满足什么条件时，方法是稳定的。

- (1) 通过代入模型方程，将方法转化：

$$y_{n+1} = \psi(h, \lambda) y_n$$

与 h 有关的函数

- (2) 进而得到扰动：

$$\delta_{n+1} = \psi(h, \lambda) \delta_n$$

- (3) 判断 $|\delta_{n+1}| \leq |\delta_n|$ ，可得出稳定的条件。

第4章 小结

- 欧拉方法
 - 显式欧拉法、隐式欧拉法、二步欧拉法、梯形公式欧拉法
 - 局部截断误差（泰勒展开，**凑！** $y(x_{n+1}) - y_{n+1} = O(h^{p+1})$ ）
- 改进的欧拉方法（鱼和熊掌兼得）
- 龙格-库塔方法：重点二阶，经典四阶
- （单步法的）收敛性与稳定性：
 - 记住收敛性判定定理，会用判定定理来判断方法的收敛性。
 - 能针对特定模型方程判断方法的稳定性。

4.5 一阶常微分方程组

$$\begin{cases} y_1' = f_1(x, y_1, y_2, \dots, y_m) \\ y_2' = f_2(x, y_1, y_2, \dots, y_m) \\ \dots\dots \\ y_m' = f_m(x, y_1, y_2, \dots, y_m) \\ y_1(x_0) = s_1, y_2(x_0) = s_2, \dots, y_m(x_0) = s_m \end{cases} \quad \longrightarrow \quad \begin{cases} Y'(x) = F(x, Y^T) \\ Y(x_0) = S \end{cases}$$

$$Y(x) = \begin{pmatrix} y_1(x) \\ y_2(x) \\ \vdots \\ y_m(x) \end{pmatrix} \quad F(x, Y^T) = \begin{pmatrix} f_1(x, Y^T) \\ f_2(x, Y^T) \\ \vdots \\ f_m(x, Y^T) \end{pmatrix} \quad S = \begin{pmatrix} s_1 \\ s_2 \\ \vdots \\ s_m \end{pmatrix}$$



$$Y(x) = \begin{pmatrix} y_1(x) \\ y_2(x) \end{pmatrix} \quad F(x, Y^T) = \begin{pmatrix} f_1(x, Y^T) \\ f_2(x, Y^T) \end{pmatrix} \quad S = \begin{pmatrix} s_1 \\ s_2 \end{pmatrix}$$

$$\begin{cases} y_1' = f_1(x, y_1, y_2) \\ y_2' = f_2(x, y_1, y_2) \end{cases} \quad \begin{cases} y_1(x_0) = s_1 \\ y_2(x_0) = s_2 \end{cases} \quad \rightarrow \quad \begin{cases} Y'(x) = F(x, Y^T) \\ Y(x_0) = S \end{cases}$$

◆ 显式欧拉法

$$Y_{n+1} = Y_n + hF(x_n, Y_n^T) \quad \begin{pmatrix} y_{1_{n+1}} \\ y_{2_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{1_n} \\ y_{2_n} \end{pmatrix} + h \begin{pmatrix} f_1(x_n, y_{1_n}, y_{2_n}) \\ f_2(x_n, y_{1_n}, y_{2_n}) \end{pmatrix}$$

◆ 隐式欧拉法

$$Y_{n+1} = Y_n + F(x_n, Y_{n+1}^T) \quad \begin{pmatrix} y_{1_{n+1}} \\ y_{2_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{1_n} \\ y_{2_n} \end{pmatrix} + h \begin{pmatrix} f_1(x_{n+1}, y_{1_{n+1}}, y_{2_{n+1}}) \\ f_2(x_{n+1}, y_{1_{n+1}}, y_{2_{n+1}}) \end{pmatrix}$$

◆ 梯形公式欧拉法

$$\begin{pmatrix} y_{1_{n+1}} \\ y_{2_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{1_n} \\ y_{2_n} \end{pmatrix} + \frac{h}{2} \left[\begin{pmatrix} f_1(x_n, y_{1_n}, y_{2_n}) \\ f_2(x_n, y_{1_n}, y_{2_n}) \end{pmatrix} + \begin{pmatrix} f_1(x_{n+1}, y_{1_{n+1}}, y_{2_{n+1}}) \\ f_2(x_{n+1}, y_{1_{n+1}}, y_{2_{n+1}}) \end{pmatrix} \right]$$



$$Y(x) = \begin{pmatrix} y_1(x) \\ y_2(x) \end{pmatrix} \quad F(x, Y^T) = \begin{pmatrix} f_1(x, Y^T) \\ f_2(x, Y^T) \end{pmatrix} \quad S = \begin{pmatrix} s_1 \\ s_2 \end{pmatrix}$$

◆ 改进的欧拉公式

$$\begin{cases} Y_p = Y_n + hF(x_n, Y_n^T) \\ Y_c = Y_n + hF(x_{n+1}, Y_p^T) \\ Y_{n+1} = (Y_p + Y_c) / 2 \end{cases}$$

$$\begin{pmatrix} y_{p1_{n+1}} \\ y_{p2_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{1_n} \\ y_{2_n} \end{pmatrix} + h \begin{pmatrix} f_1(x_n, y_{1_n}, y_{2_n}) \\ f_2(x_n, y_{1_n}, y_{2_n}) \end{pmatrix}$$

$$\begin{pmatrix} y_{c1_{n+1}} \\ y_{c2_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{1_n} \\ y_{2_n} \end{pmatrix} + h \begin{pmatrix} f_1(x_{n+1}, y_{p1_{n+1}}, y_{p2_{n+1}}) \\ f_2(x_{n+1}, y_{p1_{n+1}}, y_{p2_{n+1}}) \end{pmatrix}$$

$$\begin{pmatrix} y_{1_{n+1}} \\ y_{2_{n+1}} \end{pmatrix} = \frac{1}{2} \left[\begin{pmatrix} y_{p1_{n+1}} \\ y_{p2_{n+1}} \end{pmatrix} + \begin{pmatrix} y_{c1_{n+1}} \\ y_{c2_{n+1}} \end{pmatrix} \right]$$



$$Y(x) = \begin{pmatrix} y_1(x) \\ y_2(x) \end{pmatrix} \quad F(x, Y^T) = \begin{pmatrix} f_1(x, Y^T) \\ f_2(x, Y^T) \end{pmatrix} \quad S = \begin{pmatrix} s_1 \\ s_2 \end{pmatrix}$$

◆ 四阶龙格-库塔公式

$$\begin{cases} Y_{n+1} = Y_n + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ K_1 = hF(x_n, Y_n^T) \\ K_2 = hF(x_{n+\frac{1}{2}}, \frac{1}{2}K_1^T) \\ K_3 = hF(x_{n+\frac{1}{2}}, \frac{1}{2}K_2^T) \\ K_4 = hF(x_{n+1}, K_3^T) \end{cases}$$

$$\begin{cases} \begin{pmatrix} k_{1_1} \\ k_{2_1} \end{pmatrix} = h \begin{pmatrix} f_1(x_n, y_{1_n}, y_{2_n}) \\ f_2(x_n, y_{1_n}, y_{2_n}) \end{pmatrix} \\ \begin{pmatrix} k_{1_2} \\ k_{2_2} \end{pmatrix} = h \begin{pmatrix} f_1(x_{n+\frac{1}{2}}, \frac{1}{2}k_{1_1}, \frac{1}{2}k_{2_1}) \\ f_2(x_{n+\frac{1}{2}}, \frac{1}{2}k_{1_1}, \frac{1}{2}k_{2_1}) \end{pmatrix} \\ \begin{pmatrix} k_{1_3} \\ k_{2_3} \end{pmatrix} = h \begin{pmatrix} f_1(x_{n+\frac{1}{2}}, \frac{1}{2}k_{1_2}, \frac{1}{2}k_{2_2}) \\ f_2(x_{n+\frac{1}{2}}, \frac{1}{2}k_{1_2}, \frac{1}{2}k_{2_2}) \end{pmatrix} \\ \begin{pmatrix} k_{1_4} \\ k_{2_4} \end{pmatrix} = h \begin{pmatrix} f_1(x_{n+1}, k_{1_3}, k_{2_3}) \\ f_2(x_{n+1}, k_{1_3}, k_{2_3}) \end{pmatrix} \end{cases}$$

$$\begin{pmatrix} y_{1_{n+1}} \\ y_{2_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{1_n} \\ y_{2_n} \end{pmatrix} + \frac{1}{6} \left[\begin{pmatrix} k_{1_1} \\ k_{2_1} \end{pmatrix} + 2 \begin{pmatrix} k_{1_2} \\ k_{2_2} \end{pmatrix} + 2 \begin{pmatrix} k_{1_3} \\ k_{2_3} \end{pmatrix} + \begin{pmatrix} k_{1_4} \\ k_{2_4} \end{pmatrix} \right]$$

高阶微分方程的初值问题

m 阶常微分方程:

$$\begin{cases} y^{(m)} = f(x, y, y', y'', y''', \dots, y^{(m-1)}) \\ y(x_0) = c_0, y'(x_0) = c_1, \dots, y^{(m-1)}(x_0) = c_{m-1} \end{cases}$$

- ◆ 通过引入新的变量, 可将高阶微分方程化为一阶微分方程组的方法进行求解。

设 $y_0 = y, y_1 = y', y_2 = y'', \dots, y_{m-1} = y^{(m-1)}$


$$y_0 = y, y_1 = y', y_2 = y'', \dots, y_{m-1} = y^{(m-1)}$$

$$\begin{cases} y^{(m)} = f(x, y, y', y'', y''', \dots, y^{(m-1)}) \\ y(x_0) = c_0, y'(x_0) = c_1, \dots, y^{(m-1)}(x_0) = c_{m-1} \end{cases}$$

则 m 阶微分方程转化为如下的一阶微分方程组:

$$\begin{cases} y'_0 = y_1 \\ y'_1 = y_2 \\ \vdots \\ y'_{m-2} = y_{m-1} \\ y'_{m-1} = f(x, y_0, y_1, \dots, y_{m-1}) \end{cases}$$

初始条件为:

$$\begin{cases} y_0(x_0) = c_0 \\ y_1(x_0) = c_1 \\ \vdots \\ y_{m-1}(x_0) = c_{m-1} \end{cases}$$



将
$$\begin{cases} y'' + \sin y = 0 & (0 \leq x \leq 2) \\ y(0) = 1, \quad y'(0) = 0 \end{cases}$$

转化为一阶微分方程组，并写出经典R-K公式。

解：令 $y_0 = y$, $y_1 = y'$ ，有：

$$\begin{cases} y_0' = y_1 \\ y_1' = -\sin y_0 \end{cases}$$

$f_0(x, y_0, y_1)$

$f_1(x, y_0, y_1)$

初始条件：

$$\begin{cases} y_0(0) = 1 \\ y_1(0) = 0 \end{cases}$$



$$Y(x) = \begin{pmatrix} y_1(x) \\ y_2(x) \end{pmatrix} \quad F(x, Y^T) = \begin{pmatrix} f_1(x, Y^T) \\ f_2(x, Y^T) \end{pmatrix} \quad \begin{cases} y'_0 = y_1 \\ y'_1 = -\sin y_0 \end{cases}$$

$f_0(x, y_0, y_1)$
 $f_1(x, y_0, y_1)$

$$\begin{pmatrix} k_{0_1} \\ k_{1_1} \end{pmatrix} = h \begin{pmatrix} f_0(x_n, y_{0_n}, y_{1_n}) \\ f_1(x_n, y_{0_n}, y_{1_n}) \end{pmatrix} = h \begin{pmatrix} y_{1_n} \\ -\sin y_{0_n} \end{pmatrix}$$

$$\begin{pmatrix} k_{0_2} \\ k_{1_2} \end{pmatrix} = h \begin{pmatrix} f_0(x_{n+\frac{1}{2}}, \frac{1}{2}k_{0_1}, \frac{1}{2}k_{1_1}) \\ f_1(x_{n+\frac{1}{2}}, \frac{1}{2}k_{0_1}, \frac{1}{2}k_{1_1}) \end{pmatrix} = h \begin{pmatrix} -\frac{1}{2}h \sin y_{0_n} \\ -\sin(\frac{1}{2}hy_{1_n}) \end{pmatrix}$$

$$\begin{pmatrix} k_{0_3} \\ k_{1_3} \end{pmatrix} = h \begin{pmatrix} f_0(x_{n+\frac{1}{2}}, \frac{1}{2}k_{0_2}, \frac{1}{2}k_{1_2}) \\ f_1(x_{n+\frac{1}{2}}, \frac{1}{2}k_{0_2}, \frac{1}{2}k_{1_2}) \end{pmatrix} = h \begin{pmatrix} -\frac{1}{2}h \sin(\frac{1}{2}hy_{1_n}) \\ -\sin(-\frac{1}{4}h^2 \sin y_{0_n}) \end{pmatrix}$$

$$\begin{pmatrix} k_{0_4} \\ k_{1_4} \end{pmatrix} = h \begin{pmatrix} f_0(x_{n+1}, k_{0_3}, k_{1_3}) \\ f_1(x_{n+1}, k_{0_3}, k_{1_3}) \end{pmatrix} = h \begin{pmatrix} -h \sin(-\frac{1}{4}h^2 \sin y_{0_n}) \\ -\sin(-\frac{1}{2}h^2 \sin(\frac{1}{2}hy_{1_n})) \end{pmatrix}$$

$$\begin{pmatrix} y_{0_{n+1}} \\ y_{1_{n+1}} \end{pmatrix} = \begin{pmatrix} y_{0_n} \\ y_{1_n} \end{pmatrix} + \frac{1}{6} \left[\begin{pmatrix} k_{0_1} \\ k_{1_1} \end{pmatrix} + 2 \begin{pmatrix} k_{0_2} \\ k_{1_2} \end{pmatrix} + 2 \begin{pmatrix} k_{0_3} \\ k_{1_3} \end{pmatrix} + \begin{pmatrix} k_{0_4} \\ k_{1_4} \end{pmatrix} \right]$$